

SECURE FILE TRANSFER
MONITORING SYSTEM

Detailed Project Report

Project Type: Cybersecurity / Web-Based Monitoring System

Prepared for: Academic / Project Submission

This report explains the design, implementation, working, testing, security considerations, and future scope of the Secure File Transfer Monitoring System.

1. Abstract

The Secure File Transfer Monitoring System is a cybersecurity-focused application designed to monitor file-transfer activity and provide visibility into file operations. The system records relevant transfer information such as file name, time, source or destination details, transfer status, and other useful metadata. The purpose is to make file movement easier to observe and investigate rather than relying only on manual checking.

The project demonstrates how a simple monitoring application can support security operations by creating an activity trail, identifying unusual transfer behavior, and presenting monitoring information through a web interface. It is intended as an educational prototype that can be extended with authentication, alerting, hashing, centralized logging, and integration with SIEM platforms.

2. Introduction

File transfers are common in organizations and personal systems. Files may be copied between folders, uploaded, downloaded, or transferred between systems. Without monitoring, it can be difficult to determine what happened to a file, when the activity occurred, or whether an unexpected transfer took place.

From a cybersecurity perspective, monitoring file movement can help provide accountability and support incident investigation. This project therefore focuses on collecting file-transfer events and displaying them in an understandable form.

The project combines basic software development with cybersecurity concepts such as logging, monitoring, event analysis, access control, and security auditing.

3. Problem Statement

Organizations may have many file-transfer activities taking place every day. If these activities are not logged properly, suspicious or unauthorized movement of files may be difficult to identify. Manual monitoring is time-consuming and may not provide a consistent audit trail.

The problem addressed by this project is the need for a simple system that can record and monitor file-transfer activities and make the collected information easy to review.

4. Objectives

- Monitor file-transfer activity in a controlled environment.
- Record important details about file events for later review.
- Provide a simple web-based dashboard/interface for viewing monitoring information.
- Maintain an activity log that can support basic auditing and investigation.
- Demonstrate practical application of cybersecurity monitoring concepts.
- Create a foundation that can later be extended with alerts, hashing, authentication, and SIEM integration.

5. Scope of the Project

The project is primarily an educational prototype. Its scope includes file-event monitoring, logging, status information, and a web interface for presenting collected records. It can be used on a Windows 11 development machine in a controlled environment.

The current scope does not attempt to replace enterprise Data Loss Prevention (DLP), Endpoint Detection and Response (EDR), or a full Security Information and Event Management (SIEM) platform. Such systems require stronger controls, centralized infrastructure, authentication, secure storage, scalability, and extensive testing.

6. Technologies Used

Component	Purpose
Python	Backend logic, monitoring and processing
Flask	Lightweight web application framework
HTML/CSS	Web interface and presentation
File-system monitoring	Detecting relevant file activity
Logging	Recording monitoring events
Windows 11	Development and execution environment
Git / GitHub	Version control and project sharing

7. System Requirements

Hardware: A normal modern computer with sufficient RAM and storage for Python, the development environment, and project files.

Software: Windows 11, Python 3.x, a code editor such as Visual Studio Code, a web browser, Git, and the required Python packages.

Network: Internet access may be used to install dependencies and publish the project to GitHub; the monitoring prototype itself can run locally.

8. Proposed System Architecture

The system can be understood as four main layers:

- **File Activity Layer:** Files are created, modified, moved, or transferred within the monitored environment.
- **Monitoring Layer:** Python code observes relevant events and collects event details.
- **Logging/Data Layer:** Events are stored in a structured log or application data store.
- **Web Interface Layer:** Flask serves pages that display monitoring information to the user.

A simplified flow is: File Event → Python Monitor → Event Record → Flask Application → Web Dashboard.

9. Functional Modules

- **File Monitoring Module:** Detects selected file-system events and gathers event information.

- **Event Logging Module:** Creates records that can be reviewed later.
- **Web Application Module:** Provides browser-based access to monitoring information.
- **Dashboard / Report Module:** Presents events in a readable format.
- **Configuration Module:** Defines the directory or environment to be monitored and other application settings.

10. Working of the System

1. The user starts the Python/Flask application on the Windows 11 machine.
2. The monitoring component begins watching the configured location or file activity source.
3. When a relevant file operation occurs, the monitor captures available event details.
4. The event is converted into a structured record and written to the logging/data layer.
5. The Flask application reads the monitoring records.
6. The dashboard displays the records so the user can review file activity.
7. The logs can be used to understand normal activity and investigate unusual events.

11. Security Features and Concepts

- **Monitoring:** Provides visibility into file activity.
- **Logging:** Creates an audit trail for later review.
- **Least privilege:** The application should be run with only the permissions required for its task.
- **Input validation:** User-controlled values should be validated before being processed.
- **Secure configuration:** Secrets and credentials should not be hard-coded or uploaded to GitHub.
- **Integrity:** File hashes such as SHA-256 can be added to help verify whether a file changed.
- **Alerting:** Rules can be added to flag suspicious file movements or unexpected activity.

12. Implementation Overview

A practical implementation consists of a project directory containing the Python application, HTML templates, static resources, monitoring logic, configuration, and documentation. Flask handles web requests and renders the dashboard, while Python provides the monitoring and logging functionality.

The application should be tested locally before publication. During development, debug mode may be useful, but it should be disabled when deploying the application in a production environment.

13. Suggested Project Structure

```
Secure-File-Transfer-Monitoring-System/
├── app.py
├── monitor.py
├── requirements.txt
├── README.md
├── templates/
│   ├── index.html
│   ├── static/
│   │   ├── style.css
│   └── logs/
│       ├── events.log
│       ├── screenshots/
│       └── project screenshots
```

14. Testing

Test Case	Expected Result
Start application	Flask server starts successfully.
Open dashboard	Monitoring page loads in browser.
Create a test file	A corresponding event is recorded when configured to monitor creation.
Modify/move a test file	Relevant event is recorded if supported by the monitor.
Refresh dashboard	Newly recorded events are displayed.
Invalid input/path	Application handles the input safely without crashing.
Restart application	Application starts again and continues monitoring according to configuration.

15. Results

The completed prototype demonstrates how file-transfer or file-system activity can be monitored and presented through a simple web application. The project provides a practical example of combining programming with cybersecurity monitoring and logging concepts.

The result is useful for learning how security events can be collected, stored, reviewed, and eventually connected to larger security-monitoring workflows.

16. Advantages

- Simple and understandable architecture.
- Useful for cybersecurity learning and demonstrations.
- Provides visibility into file activity.
- Creates records that can support basic auditing.
- Can be extended without redesigning the entire concept.
- Can be maintained in GitHub using version control.

17. Limitations

- The prototype is not a replacement for enterprise DLP or EDR.
- Local logs may be lost or modified if the host is compromised.
- Advanced authentication and authorization may need to be added.
- Large-scale environments require centralized and scalable logging.
- Detection quality depends on the monitoring rules and configuration.
- Production deployment requires additional security hardening.

18. Future Enhancements

- Add user authentication and role-based access control.
- Store events in a secure database.

- Calculate and record SHA-256 hashes for monitored files.
- Add real-time alerts for suspicious activity.
- Add email or notification support.
- Create search, filtering, and date-range analysis.
- Integrate with SIEM platforms through structured logs or APIs.
- Add dashboards and charts for event trends.
- Use secure centralized logging with access controls and retention policies.

19. Conclusion

The Secure File Transfer Monitoring System demonstrates a practical cybersecurity approach to monitoring file activity. By combining Python-based monitoring, logging, and a Flask web interface, the project provides a clear view of file events and creates a foundation for security analysis.

The project also demonstrates important cybersecurity principles including visibility, auditing, monitoring, event logging, and secure software practices. With additional development, the prototype can be expanded into a more feature-rich monitoring solution.

20. References / Learning Resources

- Python documentation and standard library documentation.
- Flask documentation.
- Git and GitHub documentation.
- General cybersecurity concepts relating to logging, monitoring, incident response, and file integrity.

For the final submission, the project GitHub repository should contain the source code, README, screenshots, and this Detailed Project Report PDF.

Submission Note

Upload this PDF into the GitHub repository, preferably inside a folder named **docs** or **report**. Then copy the GitHub URL of the PDF and submit that URL wherever the form asks for the “Detailed Project Report Link.”